Studio Nx

JS 105 — ticksToSecondsWithChanges()

2026-10-14

JS 105 — `ticksToSecondsWithChanges()` (곡 중간 템포 변경을 반영한 tick→초)

항목	내용
파일	`studionx/frontend/static/js/audio/timing.js` (함수 추가, 소리와 화면을 쓰지 않는 계산 전용 모듈)
작업	S1-05 오디오 엔진 기초: 두 번째 함수 (FR-202a 곡 중간 템포 변경, FR-205 시간 단위 tick, PPQ 960)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	스케줄러(tick 을 보고 다음 소리 시각을 예약), 메트로놈, 루프 순서로 한 함수씩 진행합니다

1. 하는 일

곡 중간에 BPM 이 바뀌는 경우를 반영해 tick을 초로 바꿉니다. `projects.bpm` 은 곡의 시작(tick 0) 템포이고, `tempo_changes` 표의 변경점들을 tick 순서대로 지나가며 구간마다 초를 더합니다.

입력	결과
120 BPM, 변경 없음, 960 tick(1박)	0.5초 (앞 함수 104와 같음)
120→60 STEP 이 3840 tick 뒤, 그 위치	2초 (앞 구간 2초)
같은 곡, 그 위치에서 960 tick 뒤	3초 (60 BPM 으로 1박 = 1초 더함)
120→240 LINEAR 이 3840 tick 에 걸쳐, 그 위치	약 1.386초 (로그 적분)
tick 이 음수 (카운트인)	시작 템포 하나로 계산

`STEP` 은 변경점 위치에서 바로 바뀌고, `LINEAR` 는 바로 앞 변경점 위치부터 이 변경점까지 서서히 바뀝니다 (`DB.md` `tempo_changes.ramp`).

2. 코드

```
export function ticksToSecondsWithChanges(tick, baseBpm, changes) {
  if (typeof tick !== 'number' || !isFinite(tick)) throw new RangeError('tick 은 유한한 숫자여야 합니다: ' + tick);
  if (typeof baseBpm !== 'number' || !isFinite(baseBpm) || baseBpm <= 0) throw new RangeError('baseBpm 은 0 보다 큰 숫자여야 합니다: ' + baseBpm);

  const list = [];
  if (changes != null) {
    if (!Array.isArray(changes)) throw new RangeError('changes 는 배열이어야 합니다: ' + changes);
    for (let i = 0; i < changes.length; i++) {
      const c = changes[i];
      if (c == null) continue;
      if (typeof c.tick !== 'number' || !isFinite(c.tick) || c.tick <= 0) throw new RangeError('변경점 tick 은 0 보다 큰 유한한 숫자여야 합니다: ' + c.tick);
      if (typeof c.bpm !== 'number' || !isFinite(c.bpm) || c.bpm <= 0) throw new RangeError('변경점 bpm 은 0 보다 큰 숫자여야 합니다: ' + c.bpm);
      list.push({ tick: c.tick, bpm: c.bpm, ramp: c.ramp === 'LINEAR' ? 'LINEAR' : 'STEP' });
    }
```

```
    list.sort(function (a, b) { return a.tick - b.tick; });
  }

  if (tick <= 0) return (tick * 60) / (baseBpm * PPQ);

  let seconds = 0;
  let prevTick = 0;
  let bpm = baseBpm;
  for (let i = 0; i < list.length; i++) {
    const c = list[i];
    if (c.tick <= prevTick) continue;
    const segEnd = tick < c.tick ? tick : c.tick;
    if (c.ramp === 'LINEAR' && c.bpm !== bpm) {
      const span = c.tick - prevTick;
      const slope = (c.bpm - bpm) / span;
      const bpmAtEnd = bpm + slope * (segEnd - prevTick);
      seconds += (60 / PPQ) * (1 / slope) * Math.log(bpmAtEnd / bpm);
    } else {
      seconds += ((segEnd - prevTick) * 60) / (bpm * PPQ);
    }
    prevTick = c.tick;
    bpm = c.bpm;
    if (tick <= c.tick) return seconds;
  }
  seconds += ((tick - prevTick) * 60) / (bpm * PPQ);
  return seconds;
}
```

최신 문법으로 쓰면 (공부용)

```
export const ticksToSecondsWithChanges = (tick, baseBpm, changes) => {
  const list = (changes ?? [])
    .filter((c) => c != null)
    .map((c) => ({ tick: c.tick, bpm: c.bpm, ramp: c.ramp === 'LINEAR' ? 'LINEAR' : 'STEP' }))
    .sort((a, b) => a.tick - b.tick);
  // …나머지 계산은 같음
};
```

최신 문법	실제 코드	뜻
`changes ?? []`	`changes != null ? … : []`	값이 없으면 빈 배열을 씁니다. 실제 코드는 `if (changes != null)` 로 나누어 검사합니다
`.filter().map().sort()`	`for` 반복문	배열을 걸러 내고 바꾸고 정렬합니다
화살표 함수	`function` 선언	같은 동작입니다

3. 왜 이렇게

선택	이유
앞 함수 `ticksToSeconds` 를 고치지 않고 새 함수로 만듦	템포가 하나일 때는 104가 더 단순하고 빠릅니다. 스케줄러는 "변경점이 없으면 104, 있으면 105" 로 고르면 됩니다
구간을 차례로 더해 나감	변경점은 보통 몇 개 안 되므로 매번 처음부터 더해도 충분합니다. 나중에 자주 부르게 되면 누적 시간을 미리 계산하는 함수로 바꿀 수 있습니다

선택	이유
`LINEAR` 는 로그로 계산	BPM 이 tick 에 대해 선형으로 변할 때 초는 `1/BPM` 을 적분한 값이라 `ln` 이 나옵니다. 구간을 잘게 나누어 더하는 근사 대신 정확한 값을 씁니다
`LINEAR` 인데 시작·끝 BPM 이 같으면 상수 처리	`ln(1)=0` 이고 `slope=0` 이라 0으로 나누게 됩니다. 이 경우는 상수 구간과 같으므로 나누지 않고 그대로 더합니다
`ramp` 값이 `LINEAR` 가 아니면 `STEP` 으로	`DB.md` 의 ENUM 은 `STEP`·`LINEAR` 둘뿐입니다. 모르는 값이 와도 소리를 내는 계산이 멈추지 않게 기본값을 둡니다
정렬은 함수 안에서	호출하는 쪽이 정렬을 잊어도 결과가 틀리지 않게 합니다. 같은 tick 이 겹치면 뒤 값이 이깁니다
음수 tick 은 시작 템포로	카운트인(FR-203)은 0보다 앞이라 템포 변경이 없습니다. 104와 같은 식을 씁니다
`changes` 를 그대로 받음	데이터베이스에서 받은 `tempo_changes` 행을 그대로 넣을 수 있게 했습니다. 필요한 칸(`tick`·`bpm`·`ramp`)만 읽습니다

4. 시험 (Node에서 직접 실행, 23개 통과)

확인	결과
변경 없음·`null` 일 때 104와 같은 값(0.5초, 한 마디 2초)	✓
STEP 변경점 위치·뒤·앞 계산 (2초, 3초, 1초)	✓
STEP 두 번 (120→60→120) 구간 더하기	✓
LINEAR 전체 구간이 로그 적분 값과 일치	✓
LINEAR 구간 절반 지점(BPM 180) 값	✓
LINEAR 인데 BPM 이 같으면 상수(2초)	✓
정렬하지 않은 입력도 정렬되어 같은 결과	✓
음수 tick(−960 = −0.5초)	✓
tick · baseBpm · changes · 변경점 tick·bpm 오류 시 `RangeError`	✓
알 수 없는 `ramp` 값은 STEP 처리	✓